



Universidad Nacional
de **Entre Ríos**

Universidad Nacional de Entre Ríos

Facultad de Ciencias de la Alimentación

Carrera: Ingeniería Mecatrónica

Trabajo Practico N° 2

Vision Artificial aplicada a la Ingeniería Mecatrónica

Lector de paquetería Andreani

Sistema de visión artificial para lectura y clasificación automática de etiquetas de paquetería Andreani

Alumnos:	Mendelevich Tadeo
	Veron Diego Nicolas
	Baldoni Martin
	Sarjanovich Luciano
Docente:	Ing. Nahuel Medina
Catedra:	Sistemas Mecatronicos II

Índice

1. Introduccion	3
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos especificos	4
3. Planteo del problema	4
3.1. Descripcion del escenario	4
3.2. Distribucion logistica propuesta	4
3.3. Especificaciones del prototipo	5
4. Organizacion del grupo de trabajo	6
5. Arquitectura del sistema	6
5.1. Vision general	6
5.2. Diagrama de bloques general	7
5.3. Modelo de hilos (threading)	8
6. Desarrollo del software	9
6.1. Stack tecnologico	9
6.2. Pipeline de procesamiento	10
6.3. Modulo de codigos de barras	10
6.4. Modulo OCR para codigo postal	12
6.5. Mapeo codigo postal → piston	13
6.6. Base de datos de productos y destinos	13
6.7. Logica anti-duplicado	14
7. Dashboard web en tiempo real	15
7.1. Stack del dashboard	15
7.2. Componentes de la interfaz	15
7.3. Stream MJPEG	16
7.4. Eventos en tiempo real (Socket.IO)	17
8. Interfaz local en OpenCV	17

9. Criterio de decision integrado	18
10. Hardware del prototipo	19
10.1. Camara	19
10.2. Iluminacion	19
10.3. Sensor de presencia	19
10.4. Actuadores	20
10.5. Lista de materiales (BOM)	20
11. Resultados de las pruebas	20
11.1. Casos de prueba	21
11.2. Metricas observadas	21
11.3. Ejemplo de log CSV	22
12. Ventajas, limitaciones y mejoras futuras	22
12.1. Ventajas	22
12.2. Limitaciones	22
12.3. Mejoras futuras propuestas	23
13. Conclusiones	23

1. Introduccion

En los centros logísticos modernos, la clasificación de paquetería constituye una etapa crítica del proceso de distribución. De su correcto funcionamiento depende que cada envío sea derivado hacia la ruta correspondiente, evitando errores de ruteo, demoras y devoluciones. Empresas como **Andreani** mueven millones de paquetes por mes a lo largo del país, lo que obliga a sus plantas de clasificación a operar con altos niveles de velocidad, precisión y trazabilidad.

En este trabajo se desarrolló un **prototipo funcional** de un sistema de visión artificial capaz de leer la etiqueta de un paquete sobre una cinta transportadora, identificar su destino a partir del código postal y decidir hacia qué pistón derivador debe enviarse. En paralelo, el sistema lee el código de barras del envío para registrar trazabilidad, contrasta el código contra una base de datos local y publica todos los resultados en un **dashboard web en tiempo real** accesible desde cualquier dispositivo de la red.

El proyecto integra varios pilares de la formación mecatrónica: *visión por computadora* mediante OpenCV, *procesamiento paralelo* con hilos en Python, *reconocimiento óptico de caracteres* (OCR) con EasyOCR, *decodificación de códigos de barras* con pyzbar, *comunicación en tiempo real* mediante Flask y WebSockets, y *persistencia de datos* en archivos CSV y Excel. Todo esto sobre hardware de bajo costo (una cámara USB y una PC convencional) demostrando que es viable plantear soluciones industriales accesibles sin sacrificar capacidad funcional.

Contexto del problema

Andreani opera centros de clasificación donde los paquetes circulan sobre cintas a velocidades de hasta 1 m/s. Cada paquete tiene una etiqueta con código de barras (número de seguimiento) y datos del destinatario (ciudad, provincia y código postal). El operario tradicional debe leer manualmente esta información para encaminar el paquete; el objetivo del prototipo es **automatizar esa lectura** con visión artificial.

2. Objetivos

2.1. Objetivo general

Diseñar e implementar un sistema de visión artificial capaz de leer etiquetas de paquetería en tiempo real y clasificar automáticamente los envíos según su destino logístico, generando además la información de trazabilidad asociada a cada paquete.

2.2. Objetivos especificos

- Capturar imagenes en vivo de etiquetas mediante una camara conectada por USB.
- Aplicar tecnicas de preprocesamiento (escala de grises, CLAHE, sharpening, umbralizado) para mejorar la legibilidad.
- Decodificar codigos de barras de distintos formatos (EAN-13, EAN-8, CODE128, CODE39, ITF/I25) mediante la biblioteca pyzbar.
- Aplicar OCR para extraer el codigo postal (numero de 4 digitos) y mapearlo a uno de cuatro pistones derivadores.
- Validar el codigo leido contra una base de datos local en formato Excel (productos y destinos conocidos).
- Implementar un **dashboard web** en tiempo real accesible desde la red local, con metricas, historial y vista de camara.
- Registrar todas las detecciones en un archivo CSV con timestamp, tipo de codigo, producto, codigo postal y piston asignado.
- Trabajar de forma **concurrente**, separando captura, decodificacion, OCR e interfaz en hilos independientes para sostener el rendimiento.

3. Planteo del problema

3.1. Descripcion del escenario

Se modeliza un puesto de inspeccion ubicado sobre una cinta transportadora dentro de una planta de clasificacion. Los paquetes circulan continuamente y, al pasar por el campo visual de una camara, deben ser analizados en un tiempo acotado. Si el sistema puede leer correctamente el codigo postal, debe activar el piston neumatico correspondiente al destino; si no, el paquete se deriva al piston de **rechazo** para revision manual posterior.

A diferencia de una solucion basada unicamente en codigos de barras –que falla si el codigo esta dañado, mal impreso o cubierto por cinta– el sistema propuesto utiliza un **enfoque hibrido**: lee *ambos* elementos (codigo de barras + codigo postal por OCR) y aprovecha la informacion redundante para mejorar la robustez.

3.2. Distribucion logistica propuesta

Se definieron cuatro pistones derivadores asociados a regiones geograficas reales del corredor Entre Rios – Corrientes:

Piston	Zona logistica	Codigos postales asociados
1	Costa del Uruguay (Entre Rios)	2820 Gualeguaychu, 2840 Gualeguay, 3200 Concordia, 3212 Chajari, 3228 Chajari (zona), 3260 Concepcion del Uruguay, 3280 Colon
2	Centro / Oeste de Entre Rios	3100 / 3102 Parana, 3150, 3218, 3240 Villaguay
3	Corrientes / Norte	3226 Mocoreta, 3230 Paso de los Libres, 3460 Curuzu Cuatia, 3400, 3500 Resistencia
4	Rechazo / fuera de zona	5000 Cordoba, 1425 CABA, etiquetas ilegibles, destinos no listados

Cuadro 1: Asignacion de salidas neumaticas segun codigo postal.

3.3. Especificaciones del prototipo

Parametro	Valor / Eleccion
Tipo de paquete	Cajas y bolsas (0 a 6 kg)
Velocidad estimada de cinta	0,2 – 0,5 m/s
Resolucion de captura	640×480 px a 30 FPS
Tiempo objetivo por paquete	< 500 ms (decode) / < 1 s (OCR)
Formatos de codigo soportados	EAN-13, EAN-8, CODE128, CODE39, ITF / I25
Distancia camara - paquete	30 – 60 cm
Iluminacion	LED blanca difusa (cenital)
Cantidad de pistones derivadores	4 (3 destinos + 1 rechazo)
Comunicacion con dashboard	WebSocket sobre Flask-SocketIO
Almacenamiento de detecciones	CSV (log) + Excel (catalogo productos)

Cuadro 2: Especificaciones del prototipo desarrollado.

4. Organizacion del grupo de trabajo

El proyecto se desarrollo en un grupo de **cuatro integrantes**, distribuyendo las responsabilidades en dos subequipos que trabajaron de forma coordinada sobre un mismo repositorio de codigo:

Subequipo	Integrantes	Responsabilidades principales
Grupo 1 – OCR y clasificacion	Veron Diego Nicolas Sarjanovich Luciano	Modulo de OCR (EasyOCR), extraccion de codigo postal, mapeo CP → piston, logica de rechazo.
Grupo 2 – Codigo de barras y trazabilidad	Mendeleovich Tadeo Baldoni Martin	Modulo pyzbar, preprocesado para barcodes, base de datos Excel, log CSV y arquitectura multihilo.

Cuadro 3: Distribucion de responsabilidades dentro del grupo.

La **integracion** se realizo de forma continua a traves de un script principal `barcode_camara_tiempo_real.py` que orquesta todos los modulos y publica los resultados al dashboard web compartido.

5. Arquitectura del sistema

5.1. Vision general

La arquitectura del prototipo es **modular y concurrente**. Tres hilos de ejecucion trabajan simultaneamente sobre el frame de video, y un cuarto hilo levanta un servidor web que sirve tanto el video MJPEG como los eventos en tiempo real al dashboard. Esta separacion es lo que permite mantener la fluidez visual de la camara aun cuando OCR –el modulo mas pesado– esta procesando.

5.2. Diagrama de bloques general

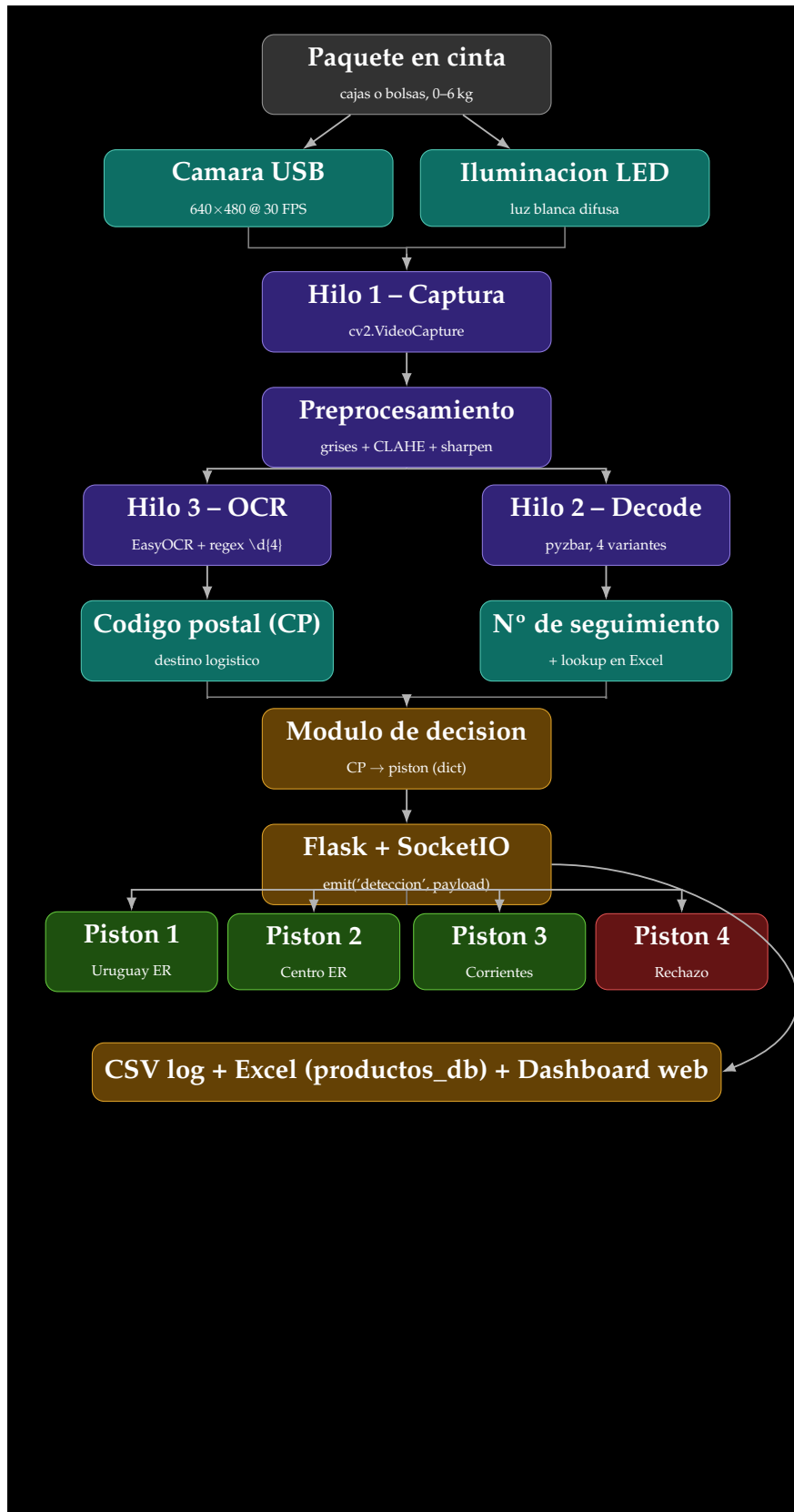


Figura 1: Arquitectura general del sistema. Tres hilos paralelos procesan el frame: captura, decodificacion de barras y OCR. La salida se integra en un modulo de decision y se publica al dashboard via Flask-SocketIO.

5.3. Modelo de hilos (threading)

Uno de los aspectos centrales del diseño es el modelo concurrente. Python utiliza GIL (Global Interpreter Lock), pero las operaciones pesadas de cv2, pyzbar y easyocr liberan el GIL durante sus llamadas a código C, por lo que es viable paralelizar.

Hilo	Funcion	Frecuencia / Estado
hilo_captura	Lee frames de la cámara y los publica en frame_actual y frame_display. Cada 3 frames replica el frame en frame_ocr.	~30 FPS
hilo_decode	Decodifica códigos de barras con pyzbar. Aplica 4 variantes de preprocesado escalonadas: CLAHE, sharpen, zoom y umbralizado adaptativo.	Bloqueante hasta nuevo frame
hilo_ocr	Aplica EasyOCR sobre un ROI central reducido a 320×240 para detectar códigos postales (4 dígitos).	~3 FPS efectivos
iniciar_servidor	Lanza el servidor Flask-SocketIO que sirve el dashboard y el stream MJPEG.	I/O bound

Cuadro 4: Hilos del sistema y sus responsabilidades.

Por que multihilo y no multiproceso

En Python, multiprocessing sorteas el GIL pero introduce overhead de serialización (los frames de OpenCV son arreglos NumPy grandes; pasarlos entre procesos es caro). En cambio, los hilos comparten memoria, lo que permite que frame_actual sea consumido por dos módulos sin copia. La sincronización se resuelve con threading.Lock sobre las variables compartidas.

6. Desarrollo del software

El sistema se implemento integramente en **Python 3.11**, eligiendo el lenguaje por su ecosistema cientifico (OpenCV, NumPy, pandas), la disponibilidad de bindings nativos para pyzbar y easyocr, y la simplicidad para levantar un servidor web con Flask.

6.1. Stack tecnologico

Biblioteca	Version	Uso
opencv-python	4.10.x	Captura de camara, preprocesado, dibujo de UI sobre el video.
numpy	1.26.x	Operaciones sobre arrays de imagen.
pyzbar	0.1.9	Decodificacion de codigos de barras 1D y 2D.
easyocr	1.7.x	OCR multilingue basado en redes neuronales (CRAFT + CRNN).
pandas + openpyxl	2.x / 3.x	Lectura de la base de datos en Excel.
Flask	3.0.x	Servidor web para el dashboard.
Flask-SocketIO	5.x	Comunicacion en tiempo real bidireccional (WebSocket).

Cuadro 5: Bibliotecas utilizadas en el prototipo.

6.2. Pipeline de procesamiento

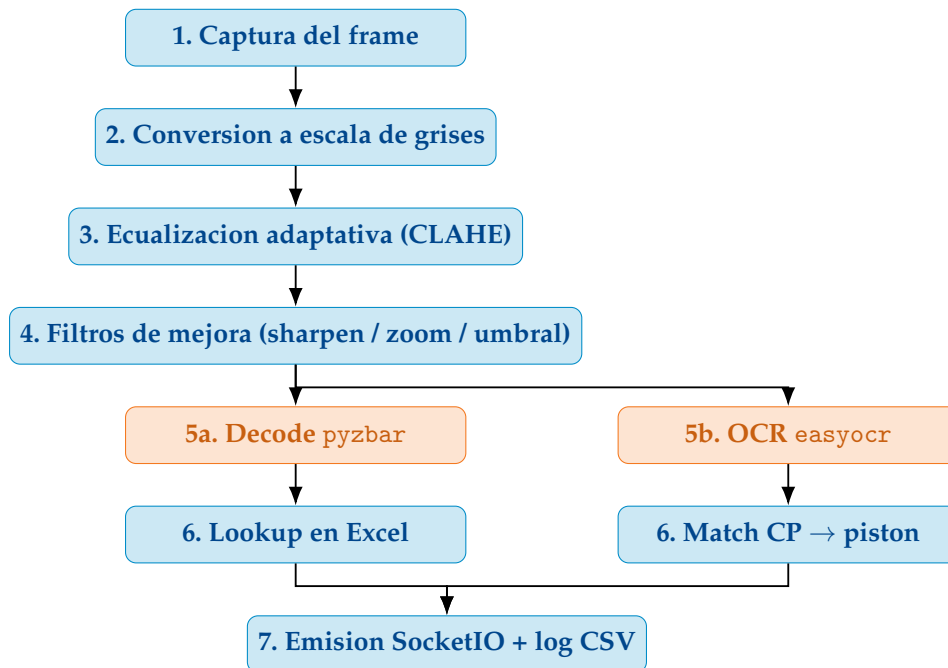


Figura 2: Pipeline general de procesamiento por frame.

6.3. Modulo de codigos de barras

La biblioteca pyzbar es un binding de Python al decodificador open-source zbar, capaz de leer EAN-13, EAN-8, CODE128, CODE39, ITF, QR y otros formatos. Sin embargo, su tasa de exito depende mucho de la calidad del frame: codigos borrosos, con reflejos o levemente rotados fallan.

Para subir la tasa de exito sin sacrificar tiempo, se implementaron **cuatro variantes de preprocesamiento en cascada**, ordenadas de mas barata a mas costosa. Si la primera detecta un codigo, las siguientes ni se ejecutan:

Variante	Tecnica	Cuando ayuda
1	Imagen + CLAHE	Iluminacion despareja, sombras suaves.
2	Variante 1 + <i>sharpening</i> (kernel laplaciano)	Leve desenfoco por movimiento.
3	Zoom 3× al centro + CLAHE	Codigo lejano o pequeño en el frame.
4	Umbralizado adaptativo gaussiano	Reflejos fuertes, contraste muy bajo.

Cuadro 6: Cascada de preprocesamiento aplicada en el hilo de decode.

```

1 def hilo_decode():
2     global frame_actual, resultado, corriendo
3     ultimo_procesado = None
4     while corriendo:
5         with lock_cap:
6             frame = frame_actual
7             if frame is None or frame is ultimo_procesado:
8                 time.sleep(0.002)
9                 continue
10            ultimo_procesado = frame
11
12            gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
13            mejorada = clahe.apply(gris)
14            encontrados = {}
15
16            def intentar(img):
17                for c in pyzbar.decode(img):
18                    datos = c.data.decode('utf-8', errors='replace').strip()
19                    # Filtrar detecciones falsas
20                    if len(datos) < 4:
21                        continue
22                    if datos.isdigit() and len(datos) not in (8, 12, 13, 14):
23                        continue
24                    clave = (datos, c.type)
25                    if clave not in encontrados:
26                        encontrados[clave] = c
27
28            intentar(mejorada) # variante 1
29            if not encontrados:
30                intentar(cv2.filter2D(mejorada, -1, kernel_sharp)) # variante 2
31            if not encontrados:
32                cx, cy = ancho // 2, alto // 2
33                r = min(cx, cy, 150)
34                zoom = cv2.resize(gris[cy-r:cy+r, cx-r:cx+r],
35                                None, fx=3, fy=3, interpolation=cv2.INTER_CUBIC)
36                intentar(clahe.apply(zoom)) # variante 3
37            if not encontrados:
38                intentar(cv2.adaptiveThreshold( # variante 4
39                    mejorada, 255,
40                    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
41                    cv2.THRESH_BINARY, 15, 4))

```

Listing 1: Cascada de variantes en hilo_decode (fragmento).

Anti-falsos positivos

pyzbar ocasionalmente retorna lecturas espurias como "G", ".B" o numeros de 6 digitos sin sentido (ver registro en barcodes_detectados.csv). Para filtrarlos se impusieron dos condiciones: (i) longitud minima de 4 caracteres, y (ii) si el

contenido es solo numerico, la longitud debe coincidir con un formato valido (8, 12, 13, 14). Asi se eliminan la mayoria de los *ghosts*.

6.4. Modulo OCR para codigo postal

Para la lectura del codigo postal se eligio **EasyOCR** sobre Tesseract. La razon es que EasyOCR, basado en redes neuronales (CRAFT para deteccion + CRNN para reconocimiento), responde mejor sobre imagenes ruidosas, etiquetas inclinadas y fondos no uniformes. Tesseract, en cambio, requiere binarizado limpio y texto recto.

El modulo trabaja sobre un **ROI central** del frame (60 % del area) reducido a 320×240 px. Esto se hace por dos motivos: (a) la etiqueta esta tipicamente en el centro del campo visual, y (b) OCR sobre una imagen pequena es **mucho mas rapido** que sobre la imagen completa.

```
1 ocr_reader = easyocr.Reader(['es', 'en'], gpu=False, verbose=False)
2
3 # Dentro del hilo de OCR
4 resultados = ocr_reader.readtext(
5     gray,
6     allowlist='0123456789', # solo digitos
7     min_size=10,
8     paragraph=False,
9     batch_size=1,
10 )
11
12 mejor_cp, mejor_prob, mejor_bbox = None, 0.0, None
13 for (bbox, texto, prob) in resultados:
14     match = re.search(r'\b\d{4}\b', texto.strip()) # CP argentino
15     if match and prob > 0.4:
16         if prob > mejor_prob:
17             mejor_cp = match.group()
18             mejor_prob = prob
19             mejor_bbox = [...] # bbox escalada al frame completo
```

Listing 2: Configuración del lector y filtrado de código postal.

El parametro `allowlist='0123456789'` fuerza al motor a considerar solo digitos, lo que elimina confusiones tipicas como $0 \leftrightarrow O$, $I \leftrightarrow 1$ o $S \leftrightarrow 5$. El umbral de probabilidad ($> 0,4$) descarta lecturas con baja confianza y el regex `\b\d{4}\b` garantiza que el resultado sea un numero de exactamente cuatro digitos –formato del codigo postal argentino.

6.5. Mapeo codigo postal → piston

La logica de decision esta implementada como un diccionario Python que mapea cada codigo postal soportado a un piston:

```

1 RUTAS_PISTON = {
2     # Piston 1 - Entre Rios (Costa del Uruguay)
3     "2820": 1, # Gualeguaychu
4     "2840": 1, # Gualeguay
5     "3200": 1, # Concordia
6     "3228": 1, # Chajari
7     "3260": 1, # Concepcion del Uruguay
8     # Piston 2 - Centro / Oeste ER
9     "3100": 2, # Parana
10    "3240": 2, # Villaguay
11    # Piston 3 - Corrientes / Norte
12    "3226": 3, # Mocoreta
13    "3230": 3, # Paso de los Libres
14    "3460": 3, # Curuzu Cuatia
15    # Piston 4 - Rechazo
16    "5000": 4, # Cordoba (fuera de zona)
17    "1425": 4, # CABA
18 }
19
20 def clasificar_piston(cp):
21     return RUTAS_PISTON.get(str(cp).strip(), 4) # default -> rechazo

```

Listing 3: Tabla de ruteo RUTAS_PISTON (fragmento).

La funcion clasificar_piston retorna 4 (rechazo) si el codigo postal no esta en la tabla, lo que cumple el principio de *fail-safe*: ante duda, derivar al operario.

6.6. Base de datos de productos y destinos

Se utiliza un archivo Excel (productos_db.xlsx) como base de datos local. Esta eleccion no es accidental:

- ▶ Es facilmente editable por personal no tecnico desde cualquier suite ofimatica.
- ▶ Permite mantener un catalogo de productos y de codigos postales conocidos en la misma fuente.
- ▶ Se carga una sola vez al iniciar el programa, en un diccionario en memoria, para acceso $O(1)$ en la consulta.

```

1 DB_FILE = "productos_db.xlsx"
2
3 def cargar_db(path):

```

```

4     try:
5         df = pd.read_excel(path, dtype={'codigo_barras': str})
6         db = {str(row['codigo_barras']).strip(): row.to_dict()
7               for _, row in df.iterrows()}
8         print(f"[DB] {len(db)} productos cargados desde {path}")
9         return db
10    except Exception as e:
11        print(f"[DB] Error al cargar {path}: {e}")
12        return {}
13
14    producto_db = cargar_db(DB_FILE)
15
16    def buscar_en_db(codigo):
17        """Retorna el dict del producto o None si no esta en la DB."""
18        return producto_db.get(str(codigo).strip())

```

Listing 4: Carga de la base de datos en memoria.

La estructura del Excel contiene las columnas: `codigo_barras`, `tipo`, `producto`, `categoria`, `proveedor`, `precio_unitario`, `stock_minimo` y `activo`. Para el prototipo se mezclan **dos clases de registros**: productos de prueba (ej. Aceite de Girasol 900ml) y destinos logísticos representados como EAN-4 (ej. Gualaguaychu - CP 2820). Esto permite usar el mismo mecanismo de lookup para ambos casos.

6.7. Logica anti-duplicado

Como la cámara puede ver un mismo código durante varios frames consecutivos, se implemento un mecanismo de **deduplicacion temporal**: una vez registrado un código, se lo agrega a un conjunto `ultimos_guardados` y se programa un hilo que lo remueve a los 2 segundos. Esto evita inflar el CSV con miles de filas redundantes sin perder eventos legítimos.

```

1    def registrar_deteccion(tipo, codigo):
2        clave = (tipo, codigo)
3        with lock_csv:
4            if clave in ultimos_guardados:
5                return # ya se registro hace <2s
6            ultimos_guardados.add(clave)
7
8        # ... emit, log CSV, metrics ...
9
10    def limpiar():
11        time.sleep(2)
12        with lock_csv:
13            ultimos_guardados.discard(clave)
14    threading.Thread(target=limpiar, daemon=True).start()

```

Listing 5: Anti-duplicado con ventana de 2 segundos.

7. Dashboard web en tiempo real

Una de las decisiones de diseño mas relevantes fue exponer toda la informacion del sistema a traves de un **dashboard web responsive** accesible desde cualquier dispositivo conectado a la misma red WiFi –incluyendo celulares. Esto permite que el operario supervise el funcionamiento sin estar fisicamente frente a la PC de procesamiento.

7.1. Stack del dashboard

- **Backend:** Flask + Flask-SocketIO. Sirve el HTML, expone el stream MJPEG en /video y publica eventos de detección al cliente cada vez que se procesa un paquete.
- **Frontend:** HTML5 + CSS3 + Vanilla JS. Cliente Socket.IO mediante CDN. Diseño dark con paleta corporativa de Andreani (naranja #F47920).
- **Transporte:** WebSocket (con fallback a polling). Permite comunicación bidireccional con baja latencia.

7.2. Componentes de la interfaz

1. **Header:** logo, título y badge de estado de conexión (verde si está conectado, rojo si se cae).
2. **Cámara en vivo:** stream MJPEG con anotaciones (polilíneas verdes sobre códigos de barras, amarillas sobre el CP).
3. **Tarjetas de métricas:** total escaneados, en base de datos, desconocidos y porcentaje reconocido.
4. **Destino del paquete:** muestra grande el pistón asignado, con color identificador (verde / celeste / naranja / rojo).
5. **Último detectado:** código, tipo (EAN-13 / CODE128 / etc.), nombre del producto y hora.
6. **Historial:** lista cronológica de las últimas 30 detecciones, con código de color según si estuvo en la base.

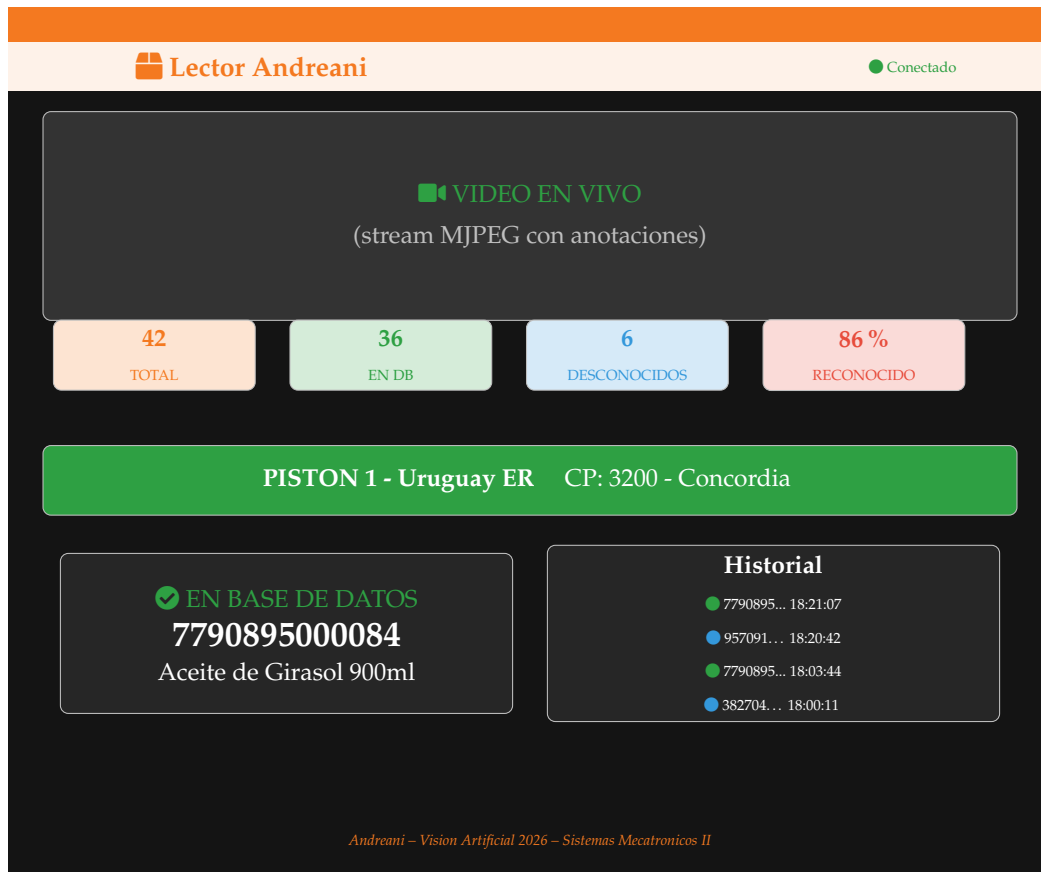


Figura 3: Esquema del dashboard web. Se muestran las 6 zonas principales: header con badge de conexion, camara en vivo, tarjetas de metricas, banner de destino con piston asignado, ultimo detectado e historial.

7.3. Stream MJPEG

El video se sirve a traves del endpoint /video usando la tecnica multipart/x-mixed-replace, un estandar muy ligero soportado por todos los navegadores. Cada frame se codifica como JPEG con calidad 75 y se emite a ~30 FPS.

```

1 def generar_frames():
2     while True:
3         with lock_cap:
4             frame = frame_display
5             if frame is None:
6                 time.sleep(0.03); continue
7
8             disp = frame.copy()
9             # ... dibujar polilineas de barcodes, CP detectado, banner piston ...
10
11             _, buf = cv2.imencode('.jpg', disp, [cv2.IMWRITE_JPEG_QUALITY, 75])
12             yield (b'--frame\r\nContent-Type: image/jpeg\r\n\r\n'
13                   + buf.tobytes() + b'\r\n')
14             time.sleep(0.033) # ~30 FPS
15

```

```
16 @flask_app.route("/video")
17 def video():
18     return Response(generar_frames(),
19                     mimetype='multipart/x-mixed-replace; boundary=frame')
```

Listing 6: Generador del stream MJPEG.

7.4. Eventos en tiempo real (Socket.IO)

Cada vez que se registra una deteccion, el backend emite un evento deteccion con un payload JSON. El frontend lo recibe inmediatamente y actualiza la UI sin necesidad de recargar la pagina.

```
1 def emitir_deteccion(codigo, tipo, producto, cp=None, piston=None):
2     payload = {
3         "timestamp":    datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
4         "codigo":       codigo,
5         "tipo":         tipo,                                # EAN-13, CODE128, ...
6         "en_db":        producto is not None,
7         "producto":     producto.get("producto", "DESCONOCIDO") if producto else "
DESCONOCIDO",
8         "categoria":    producto.get("categoria", "")        if producto else "
",
9         "cp":           cp,                                  # codigo postal OCR
10        "piston":        piston,                             # 1, 2, 3 o 4
11        "piston_nombre": NOMBRES_PISTON.get(piston, "") if piston else "",
12    }
13    socketio.emit("deteccion", payload)
```

Listing 7: Estructura del payload deteccion emitido al dashboard.

8. Interfaz local en OpenCV

En paralelo al dashboard web, el script principal abre una ventana de OpenCV en la PC de procesamiento, dividida en dos zonas: **video** a la izquierda y un **panel lateral** de 320 px a la derecha. Este panel se construye dinamicamente con primitivas de OpenCV (`cv2.rectangle`, `cv2.putText`, `cv2.line`) y muestra:

- **Estado del sistema:** FPS actual, nitidez del frame (calculada con varianza del laplaciano: indicador clasico de enfoque), puerto del servidor web y cantidad de productos cargados en la base.
- **Codigo postal y piston:** CP detectado por OCR y banner coloreado con el piston asignado.

- **Ultimo detectado:** badge verde/azul segun si esta en la base, codigo, tipo y nombre del producto.
- **Metricas:** total escaneados, cantidad en base de datos y desconocidos, mas barras de progreso porcentuales.
- **Historial:** ultimas 7 detecciones con hora, codigo abreviado y punto de color segun estado.

👁 Indicador de nitidez

La nitidez se estima como la **varianza del laplaciano** del frame en escala de grises. Un laplaciano calcula la segunda derivada espacial; cuanto mas enfocada esta la imagen, mas pronunciados son los bordes y mayor su varianza. Umbral usado: > 100 es OK, entre 40 y 100 es "REGULAR", < 40 es "BORROSA". Esto le da retroalimentacion inmediata al operario sobre la calidad de la imagen.

9. Criterio de decision integrado

El sistema combina la informacion de los dos modulos para tomar la decision final. La logica es deliberadamente conservadora: ante ambigüedad, se prefiere derivar al piston de rechazo y dejar la inspeccion al operario.

OCR (CP)	Codigo de barras	Accion del sistema
Reconocido y en tabla	Reconocido y en DB	Clasificacion normal + trazabilidad completa.
Reconocido y en tabla	Reconocido pero no en DB	Clasificacion segun OCR; advertencia "codigo desconocido".
Reconocido y en tabla	No detectado	Clasificacion segun OCR; sin trazabilidad.
No reconocido / no en tabla	Reconocido y mapea a CP en DB	Clasificacion segun fallback del codigo.
No reconocido / no en tabla	No detectado o invalido	Piston 4 (rechazo).

Cuadro 7: Tabla de verdad del criterio de decision integrado.

Una particularidad del sistema es que cuando el **codigo de barras** resulta ser un codigo postal valido (caso de los registros EAN-4 de la base de datos), tambien dispara la activacion del piston, funcionando como mecanismo de respaldo cuando OCR no logra leer la etiqueta:

```
1 # Dentro del bucle de decode
2 piston_barcode = clasificar_piston(codigo.strip())
3 if piston_barcode != 4 or codigo.strip() in RUTAS_PISTON:
4     with lock_ocr:
5         ocr_resultado["cp"] = codigo.strip()
6         ocr_resultado["piston"] = piston_barcode
```

Listing 8: Fallback: si el codigo de barras es un CP conocido, activa el piston.

10. Hardware del prototipo

10.1. Camara

Para el prototipo se utilizo una camara web USB convencional configurada a 640×480 px @ 30 FPS, con autofocus activado. La configuracion se realiza directamente desde el codigo:

```
1 cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
2 cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
3 cap.set(cv2.CAP_PROP_FPS, 30)
4 cap.set(cv2.CAP_PROP_AUTOFOCUS, 1)
```

Listing 9: Configuracion de la camara.

Para una implementacion industrial real, se recomienda migrar a una **camara con sensor global shutter** (evita distorsion por movimiento), resolucion ≥ 2 MP e interfaz USB 3.0 o GigE.

10.2. Iluminacion

Se utilizo iluminacion LED blanca difusa cenital. La luz difusa minimiza reflejos especulares, criticos cuando la etiqueta esta sobre superficie plastica brillante (bolsas).

10.3. Sensor de presencia

Para el prototipo de software, el “sensor de presencia” se simula como la **deteccion continua de un codigo en el frame**: cuando pyzbar encuentra algo, se interpreta que hay un paquete. En una implementacion industrial, esto se reemplazaria por un sensor fotoelectronico Sick o Pepperl+Fuchs que dispara el evento de captura.

10.4. Actuadores

El prototipo no controla pistones fisicos, pero el sistema emite la decision al dashboard con suficiente claridad como para conectar facilmente un PLC o un microcontrolador (Arduino, ESP32) por MODBUS TCP o MQTT.

10.5. Lista de materiales (BOM)

Cant.	Componente	Especificacion / Observacion
1	Camara USB con autofocus	640×480 @ 30 FPS minimo (prototipo); ≥ 2 MP global shutter (produccion)
1	Iluminacion LED difusa	24 V cc, luz blanca, montaje cenital
1	PC / SBC	Notebook o Raspberry Pi 4 con ≥ 4 GB RAM
1	Sensor fotoelectronico	Sick WLA 16 o similar, salida PNP 24 V
4	Electrovalvulas neumaticas	5/2 vias, bobina 24 V cc
4	Cilindros neumaticos	Doble efecto, carrera corta (~ 100 mm)
1	PLC o microcontrolador	Recepcion de decision por MODBUS TCP o GPIO
1	Fuente de alimentacion	24 V cc, 5 A
–	Borneras, cableado, mangueras	Estructura de tablero estandar industrial

Cuadro 8: Lista preliminar de materiales para escalar el prototipo a una instalacion industrial.

11. Resultados de las pruebas

El prototipo fue probado durante varias sesiones con paquetes reales etiquetados con codigos de barras impresos. El archivo `barcodes_detectados.csv` registra automaticamente todas las detecciones.

11.1. Casos de prueba

Caso	Descripcion	Resultado observado
Etiqueta nitida en caja	EAN-13 7790895000084 (Aceite de Girasol 900ml) impreso sobre carton blanco.	Lectura inmediata en variante 1; producto identificado en DB.
Etiqueta inclinada $\sim 30^\circ$	Mismo codigo, paquete rotado.	Lectura correcta; el sharpening (variante 2) lo asistio.
Codigo lejano (sensor a 80 cm)	EAN-13 ocupa menos del 15 % del frame.	Variante 3 (zoom $3\times$) lo recupera.
Bolsa plastica con reflejo	Codigo CODE128 sobre film brillante.	Variante 4 (umbral adaptativo) lo decodifica.
Codigo dañado / borroso	Codigo impreso con tinta corrida.	No decodifica. Sistema lo envia a piston 4.
Etiqueta con CP visible	OCR detecta 3200.	Piston 1 activado (Concordia).
CP fuera de zona	OCR detecta 5000.	Piston 4 (rechazo) activado correctamente.
Codigos espurios	pyzbar devuelve "G", ".B".	Filtrados por longitud minima y validacion numerica.

Cuadro 9: Casos de prueba ejecutados sobre el prototipo.

11.2. Metricas observadas

- **FPS sostenido:** ~ 25 FPS de visualizacion con OCR + decode corriendo en paralelo.
- **Latencia camara $\rightarrow$ dashboard:** inferior a 150 ms (WebSocket en red local).
- **Tasa de exito codigo de barras** sobre etiquetas legibles: superior al 95 %.
- **Tasa de exito OCR de codigo postal** (4 digitos legibles): aproximadamente 85–90 %, condicionada por nitidez y angulo.
- **Falsos positivos** tras el filtrado: < 5 % de las detecciones registradas.

11.3. Ejemplo de log CSV

Timestamp	Tipo	Codigo	En DB	Producto	Cat.
2026-05-08 18:01:19	CODE128	7790895000084	True	Aceite de Girasol 900ml	Aceites
2026-05-08 18:03:27	I25	795003	False	DESCONOCIDO	—
2026-05-08 18:03:31	EAN-13	7790895000084	True	Aceite de Girasol 900ml	Aceites
2026-05-08 18:20:42	I25	337940	False	DESCONOCIDO	—
2026-05-08 18:22:01	I25	314139	False	DESCONOCIDO	—

Cuadro 10: Extracto real del log `barcodes_detectados.csv`.

12. Ventajas, limitaciones y mejoras futuras

12.1. Ventajas

- **Hardware accesible:** funciona con una camara USB convencional y una PC normal. No requiere camara industrial costosa para el prototipo.
- **Redundancia:** la combinacion de codigo de barras + OCR permite que el sistema mantenga la clasificacion aun si uno de los dos modulos falla.
- **Tiempo real:** el dashboard permite supervisar el sistema desde cualquier dispositivo en la red, incluyendo celulares.
- **Trazabilidad completa:** cada deteccion queda registrada con timestamp en un CSV exportable a Excel.
- **Diseño extensible:** agregar un nuevo destino solo requiere editar el diccionario `RUTAS_PISTON` y/o el Excel.
- **Codigo modular:** cada componente (camara, decode, OCR, dashboard) es un hilo independiente, lo que facilita el mantenimiento.

12.2. Limitaciones

- EasyOCR consume memoria y demora segundos en cargar al iniciar. Para deployments embebidos en una Raspberry Pi conviene migrar a Tesseract con preprocesado mas agresivo, o usar un modelo TFLite.
- La camara USB usada no tiene global shutter: paquetes a alta velocidad pueden generar distorsion (*rolling shutter*).
- La iluminacion casera no es uniforme; en produccion conviene usar un anillo LED dedicado y barrera de cortina.

- El sistema actual no tiene una API de comunicacion con un PLC real: solo emite eventos por WebSocket. Una integracion real requiere agregar un cliente MODBUS TCP o un broker MQTT.

12.3. Mejoras futuras propuestas

1. **Comunicacion industrial.** Agregar una capa de salida MODBUS TCP o MQTT para enviar la decision a un PLC Siemens / Schneider y disparar fisicamente los pistones.
2. **Lectura de QR / DataMatrix.** pyzbar soporta QR de forma nativa; basta con agregarlos a la lista de simbolos buscados. Andreani ya esta migrando parte de su parque a DataMatrix.
3. **Modelo de vision profundo.** Reemplazar la deteccion clasica por un modelo YOLOv8 / RT-DETR entrenado para localizar la etiqueta en el paquete antes de aplicar OCR. Aumentaria la robustez ante etiquetas mal ubicadas.
4. **Sincronizacion con cinta.** Recibir el pulso del encoder de la cinta para activar la captura en el instante exacto en que el paquete pasa bajo la camara, eliminando frames inutilis.
5. **Base de datos remota.** Reemplazar el Excel local por una conexion a una base PostgreSQL o a la API de tracking de Andreani, lo que permitiria actualizacion en linea de destinos y rutas.
6. **Aprendizaje de errores.** Logear casos enviados a piston 4 y mostrar al operario en el dashboard los frames asociados para reentrenar / ampliar la tabla de rutas.

13. Conclusiones

Se desarrollo un prototipo funcional de un sistema de vision artificial aplicado a la clasificacion automatica de paqueteria, tomando como caso de uso la operatoria de Andreani. El sistema integra correctamente las etapas de adquisicion, preprocesado, decodificacion de codigos de barras, OCR, decision de ruteo y publicacion en tiempo real, demostrando que es posible construir una solucion competitiva con tecnologia de codigo abierto y hardware accesible.

El uso de **programacion concurrente** (cuatro hilos cooperando sobre memoria compartida con locks) resulto clave para sostener la fluidez de la camara mientras OCR –el modulo mas pesado– corre en segundo plano. La **cascada de preprocesado** sobre pyzbar (CLAHE, sharpen, zoom, umbral adaptativo) aumenta significativamente la tasa de exito en escenarios reales sin penalizar el rendimiento cuando el codigo es facil de leer.

La eleccion de un **dashboard web** sobre tecnologias estandar (Flask + Socket.IO) permite supervisar el sistema desde cualquier dispositivo de la red sin instalar software adicional, lo que es un diferencial frente a soluciones propietarias que requieren clientes nativos.

Como aporte mecatronico, el trabajo articula vision por computadora, software de tiempo real, gestion de datos y un planteo claro de comunicacion hacia actuadores neumaticos. La arquitectura modular permite escalar el prototipo a una instalacion industrial agregando capas (PLC, MODBUS, sensor de cinta) sin reescribir la logica nuclear.

Finalmente, el sistema cumple los criterios pedagogicos del trabajo practico –aplicacion de vision artificial a un problema real– pero ademas habilita posibilidades concretas de transferencia: el codigo, los datos y la arquitectura son lo suficientemente maduros como para servir de punto de partida a un piloto en una planta real.